

Linux capabilities

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Las capabilities parten los privilegios de root en piezas concretas que se asignan a un binario sin darle SUID completo. Es una version mas granular del suid: en vez de 'corre como root', es 'puede hacer esta cosa de root'. Algunas capabilities permiten escalar directamente.

Se usan para que un binario haga una tarea privilegiada (abrir un puerto bajo, cambiar de uid) sin ser root del todo. Pero cap_setuid permite a un interprete (python, perl) cambiar su uid a 0, y cap_dac_read_search permite leer cualquier fichero. Mal puestas, son root.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeracion (enum-privesc-linux), justo despues de SUID. getcap recorre el sistema buscandolas.

Cómo identificarla

- getcap lista un binario con cap_setuid+ep (escalada directa via python/perl).
- cap_dac_read_search+ep: lectura de /etc/shadow y cualquier fichero protegido.
- cap_setuid sobre un interprete no estandar puesto a proposito en el reto.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

getcap -r / 2>/dev/null y busca cap_setuid o cap_dac_read_search en la salida.

Evidencia esperada: Una linea 'binario = cap_setuid+ep' sobre un interprete (python/perl); tras el abuso, euid=0.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
getcap -r / 2>/dev/null
```

Objetivo y contexto: Lista todos los binarios con capabilities asignadas. -r recorre recursivamente; el ruido de permiso denegado se oculta con 2>/dev/null.

Resultado esperado: Líneas 'binario = cap_xxx+ep'. cap_setuid -> escalada directa (GTF0Bins).

cap_dac_read_search -> lees /etc/shadow. Ignora los del sistema base.

Capa 2: prueba documentada

```
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Objetivo y contexto: Si python tiene cap_setuid, setuid(0) cambia tu uid a root y luego lanzas una shell que ya corre como root. Es la receta de GTF0Bins para esa capability.

Resultado esperado: Prompt de root y id con uid=0. Cambia python por el interprete concreto que getcap marco con cap_setuid.

Anatomía de los payloads

La primera prueba es `getcap -r / 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una línea 'binario = cap_setuid+ep' sobre un interprete (python/perl); tras el abuso, euid=0.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Recorre el sistema con `getcap -r` buscando capabilities en binarios.
- Identifica la capability y el binario en GTF0Bins (sección Capabilities).
- Si es cap_setuid en python/perl, fuerza setuid(0) y lanza una shell.
- Confirma uid=0 y recoge la flag.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `getcap -r /` no devuelve ningún binario fuera de los estándar del sistema.
- Las capabilities presentes no incluyen cap_setuid ni cap_dac_read_search (son inofensivas para privesc).

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Permisos finos que dan a un binario poderes de root concretos; cap_setuid sobre un interprete es escalada directa. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(capabilities\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.